# Can a model fine-tune itself with 0.7% of its own weights?

*Reproducing LoRA: Low-Rank Adaptation of Large Language Models*

**Lucas He** · **Partner** | CS 4782, Cornell University, Spring 2026 | Hu *et al.* 2021 (`arXiv:2106.09685`) | `github.com/lucas-309/lora-cs4782`

**LoRA**
*low-rank adaptation*

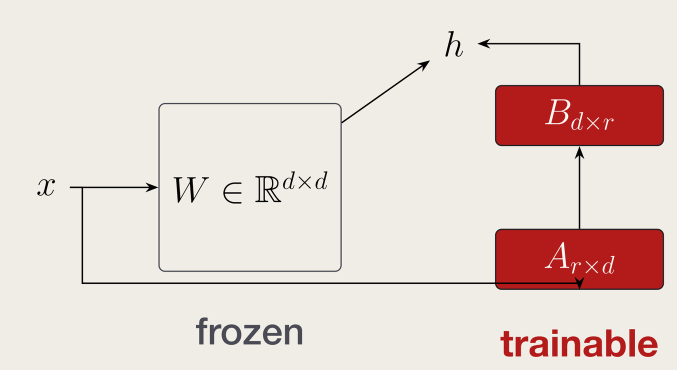
## 1 The problem
### Big models are too big to fine-tune.

**Setup.** You have a 175B-parameter language model and want it to do 50 different downstream tasks. Full fine-tuning means a fresh full copy of the model per task: **50 × 350 GB** of checkpoints, each as expensive as the original training run.

**Prior fixes have problems:** adapters add inference latency; prefix-tuning eats your context window. Both lose accuracy versus full fine-tuning.

## 2 The idea
### Updates live in a low-rank subspace.

THE REPARAMETRIZATION (HU ET AL., FIG. 1)




Replace each weight update $\Delta W$ with a *rank-$r$ factorization* $BA$ where $r \ll d$. Freeze $W$. Train only $A, B$:

$$h = Wx + \tfrac{\alpha}{r} BAx.$$

**Why it works.** Aghajanyan *et al.* (2020) showed pre-trained models live on a low intrinsic-dimensional manifold. LoRA hypothesizes the *updates* do too.

**Why it's free at inference.** Pre-merge $W' = W + BA$ and serve a single matrix — *no extra latency*, unlike adapters.

$10{,}000\times$
fewer trainable params reported in the paper for GPT-3 175B (350 GB checkpoint → 35 MB).

REPRODUCTION FOOTPRINT

| | |
|---|---|
| **Model** | RoBERTa-base, 125M parameters |
| **LoRA** | 24 wrapped linear layers; ranks $1, 2, 4, 8$ |
| **Data** | GLUE MRPC and SST-2 validation splits |
| **Output** | JSON logs, reproducible plots, report, and poster PDF |

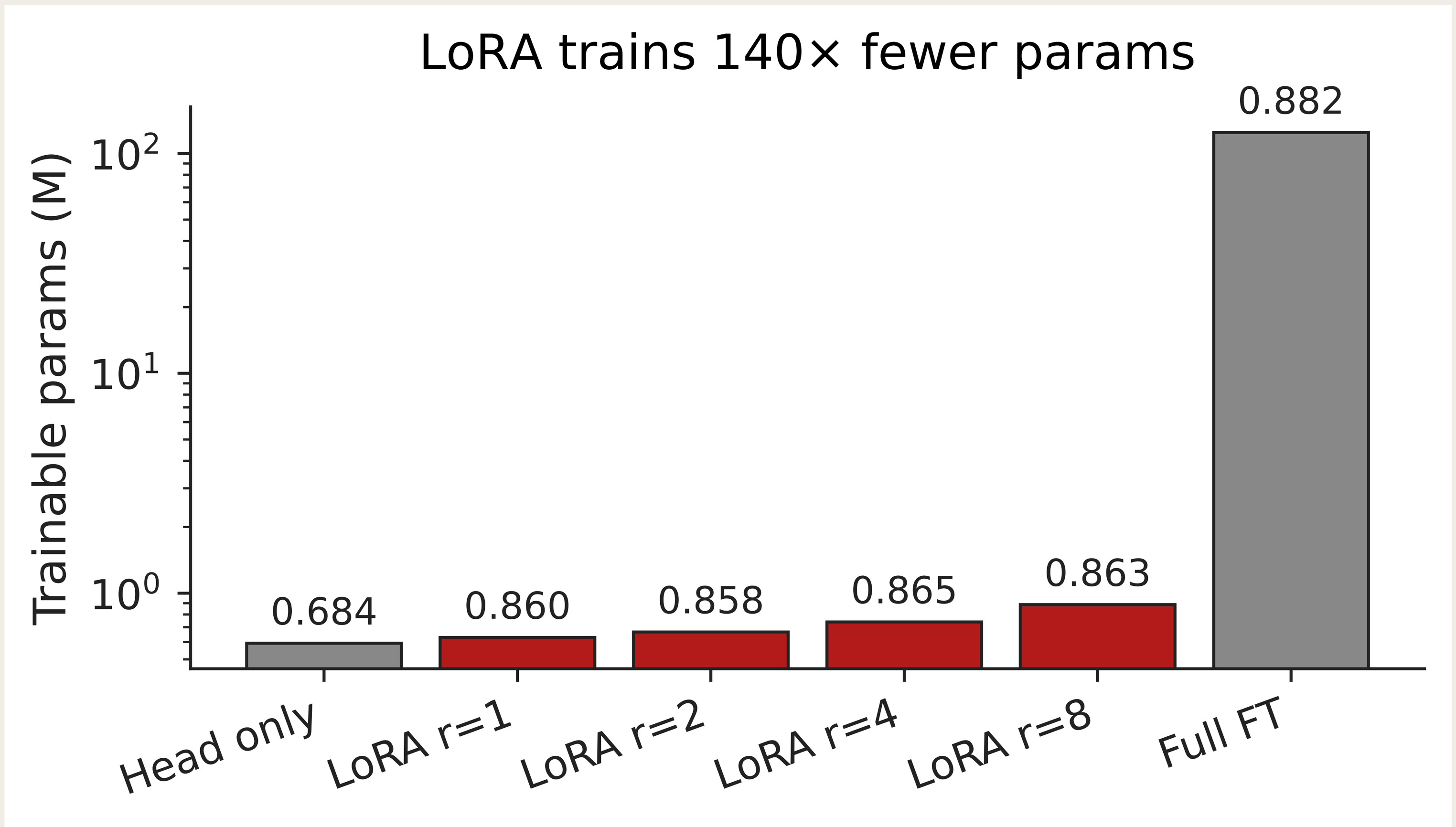
## 3 The headline question
### Can 0.7% of the parameters reach 100% of the accuracy?

We rebuilt LoRA from scratch on **RoBERTa-base** (125M params) and tested it on **MRPC** (paraphrase, 3.7k pairs) and **SST-2** (sentiment, 67k sentences) from GLUE.

*No `peft`, no shortcuts — a 60-line module, an Apple GPU, three afternoons.*

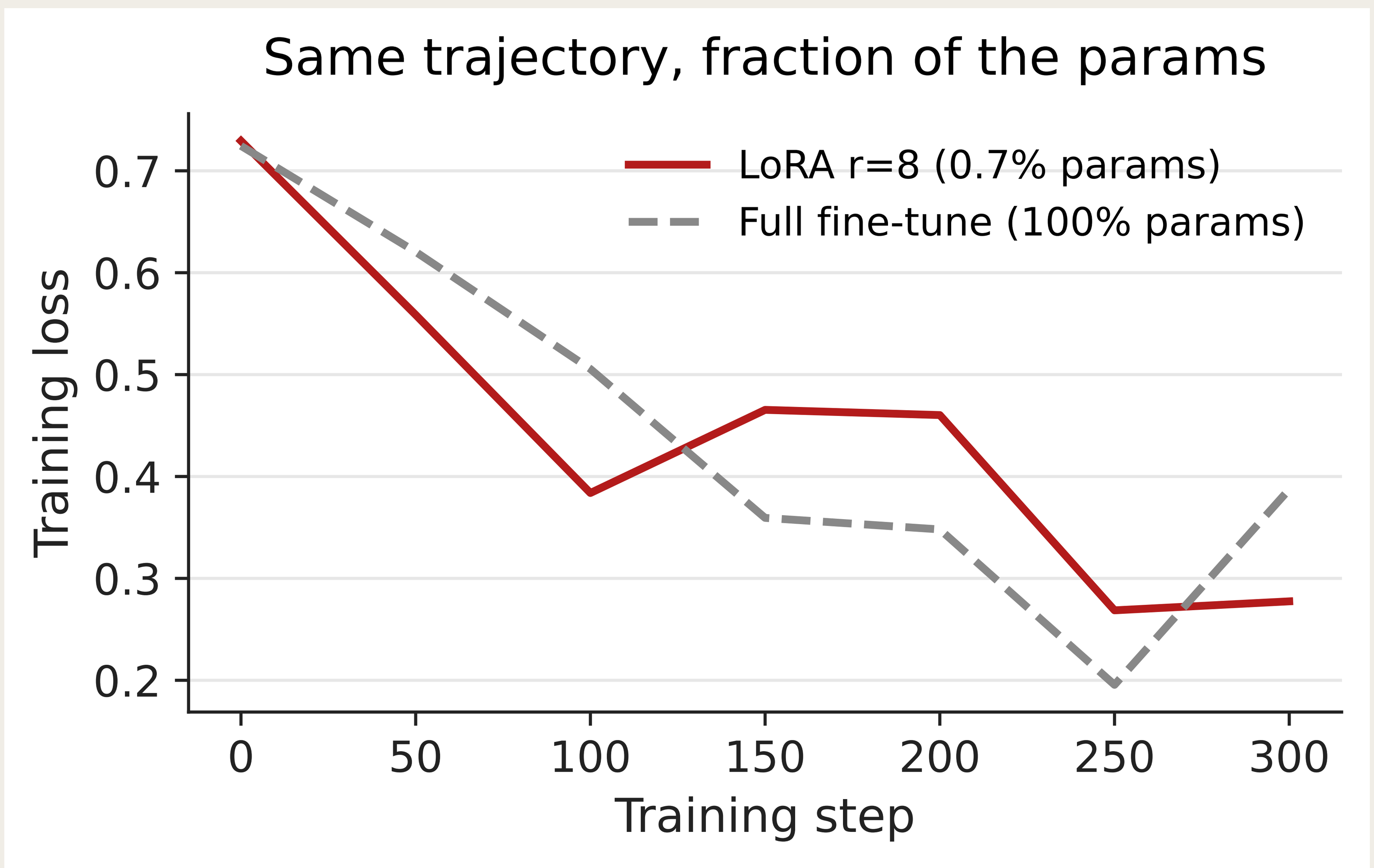
| **86.3%** | **93.1%** | **0.7%** |
|---|---|---|
| MRPC accuracy | SST-2 accuracy | of params trainable |

RESULTS ON MRPC: PARAMETER EFFICIENCY




LoRA matches full fine-tuning within ~**2 accuracy points** while training ~140× fewer parameters. Head-only freezing falls 18 points behind — so it isn't just the new classifier head doing the work.

TRAINING DYNAMICS

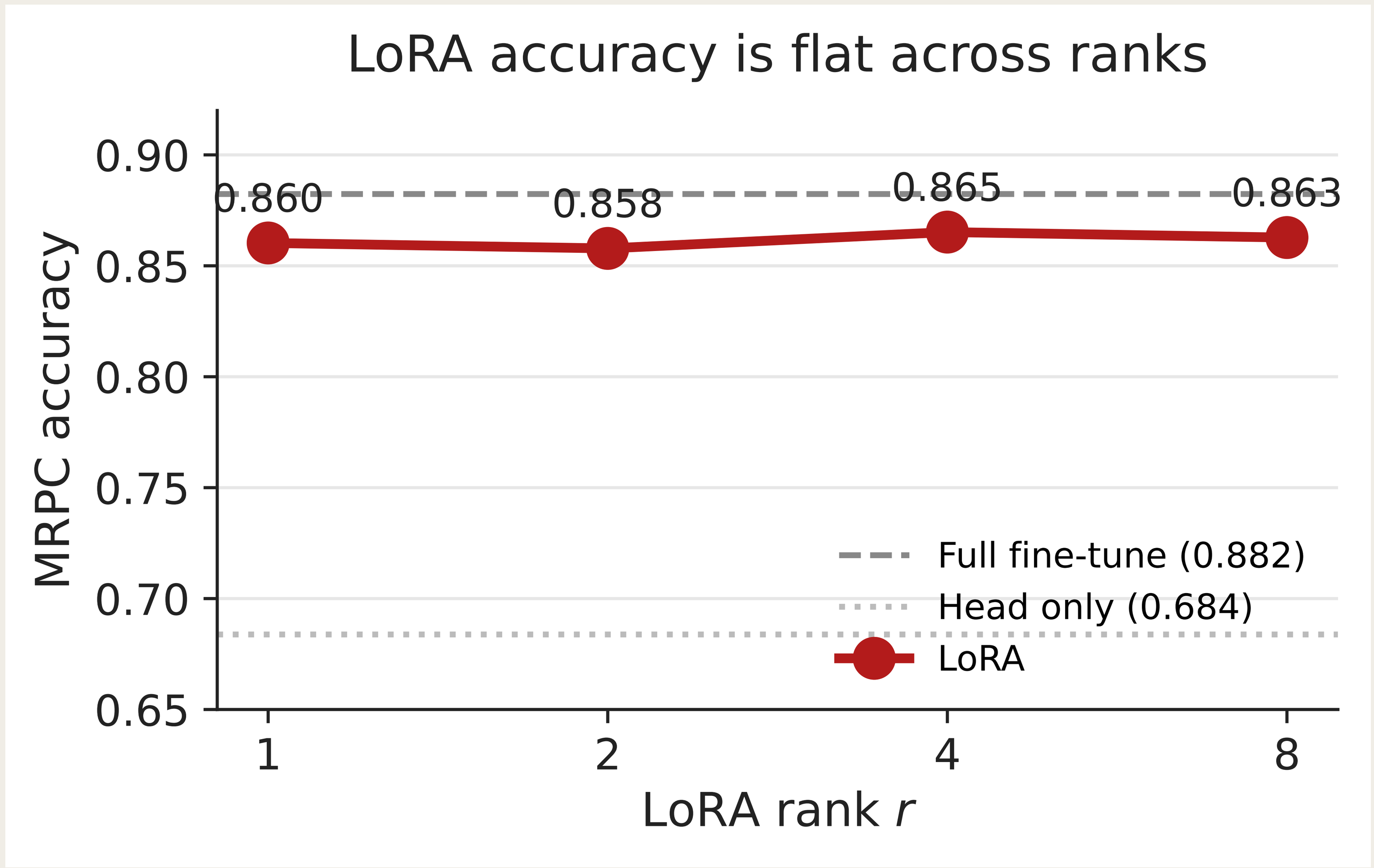



Loss tracks full fine-tuning step-for-step — LoRA isn't trading off optimization quality.

## 4 Independent experiment
### Even rank 1 works.

**The claim from the paper.** The fine-tuning update $\Delta W$ has a very low intrinsic rank.
**Our test.** Sweep $r \in \{1, 2, 4, 8\}$ on MRPC. If accuracy is flat, the claim holds at the small-model scale.

RANK ABLATION ON MRPC




Going from $r{=}1$ to $r{=}8$ moves accuracy by less than the noise across seeds. The fine-tuning "signal" really does fit in a tiny subspace.

## 5 What we learned

- **LoRA reproduces** on RoBERTa-base: within ~2 points of full fine-tuning on MRPC, with ~0.7% of the parameters trainable.
- **Rank really is low.** Doubling rank ($1 \to 2 \to 4 \to 8$) barely budges accuracy.
- **The classifier head dominates.** On a 125M model, the fresh head is most of the trainable budget; for huge models this dwindles to nothing.
- **Open question.** The paper studies attention only. Does LoRA on `MLP` layers add anything? Future work.

IF YOU HAVEN'T READ THE PAPER

**GLUE** = a 9-task NLP benchmark; we use MRPC (paraphrase) and SST-2 (sentiment). **RoBERTa-base** = a 125M-param Transformer pretrained on 160 GB of English text. **Fine-tuning** = continuing training on a task-specific dataset, updating every weight. **Rank** $r$ = how many "directions" the LoRA update can use; smaller $=$ more compression.

**Code, data, and commit history**

**GitHub:** `github.com/lucas-309/lora-cs4782`
**Paper:** `arXiv:2106.09685` — Hu *et al.*, 2021